## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

## Integration Design

Integrations must document ownership, credentials, rate limits, retries, failure handling, logging, data mapping, and reconciliation steps. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Data Modeling

Data models should protect integrity, support reporting, avoid unnecessary duplication, and include migration plans for production changes. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## Maintainability

Systems should be designed so another Code 5 Developers engineer can understand, run, troubleshoot, and extend them. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## System Design

System design must identify user roles, data flows, integrations, permission boundaries, scalability needs, reporting requirements, and operational constraints. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

## API Design

APIs should use clear resource names, predictable status codes, validation errors, authentication requirements, versioning rules, and example payloads. This page is part of the Code 5 Developers knowledge library and supports software development, systems development, maintenance, and technology operations.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.

Operational note: use the documented process, keep client decisions visible, protect client data, and escalate delivery, security, or production risks before they become client-impacting issues.